

智能识别系统设计

上课地点：南区计算机大楼323

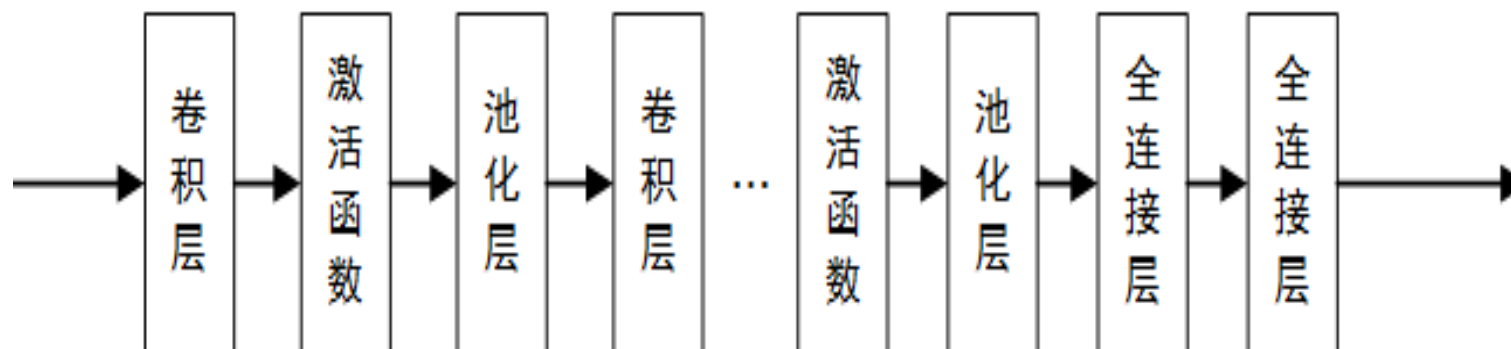
深圳大学
计算机与软件学院

- 卷积神经网络概述
- CNTK+Keras深度编程环境搭建
- CIFAR图像分类网络学习
- 实验4

● 卷积神经网络概述

生物神经元所能处理信息的复杂程度被称为神经元的感受野，而神经元对于信息位置与方向变化不敏感的特性被称为平移不变性，卷积神经网络正是根据生物神经系统的这些特性而提出的神经网络模型。

下图展示了传统卷积神经网络的大致结构，此类网络主要对输入数据进行逐层交替的卷积和池化操作。



- 卷积神经网络概述

一、卷积：

卷积是一类对矩阵的线性运算方式，包含卷积操作的网络层被称为卷积层，其用来对输入图片进行特征提取，输出相应图片的特征图。

 ● 卷积神经网络概述

假设现有一个大小为 $M \times N$ 的矩阵 $\mathbf{A}$ 与一个大小为 $m \times n$ 的矩阵 $\mathbf{B}$,且 $M \geq m, N \geq n$,则矩阵 $\mathbf{A}$ 中第 i 到第 $i + m - 1$ 行、第 j 到第 $j + n - 1$ 列元素所组成的子区域与矩阵 $\mathbf{B}$ 的卷积结果 c_{ij} 为:

$$c_{ij} = \sum_{u=i}^{i+m-1} \sum_{v=j}^{j+n-1} a_{uv} b_{u-i+1, v-j+1}$$

其中, $1 \leq i \leq M - m + 1$ 且 $1 \leq j \leq N - n + 1$

● 卷积神经网络概述

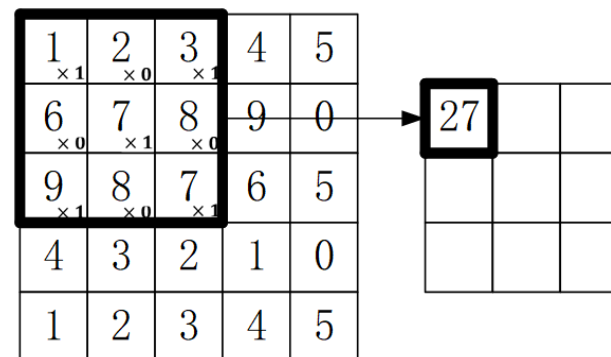
以下左图中 5×5 大小的矩阵 A 与 3×3 大小的矩阵 B 进行卷积操作为例，对于矩阵 A 中第1到第3行、第1到第3列元素所组成的子区域，利用矩阵 B 进行卷积所得到的结果为：

1	2	3	4	5
6	7	8	9	0
9	8	7	6	5
4	3	2	1	0
1	2	3	4	5

矩阵A

1	0	1
0	1	0
1	0	1

矩阵B



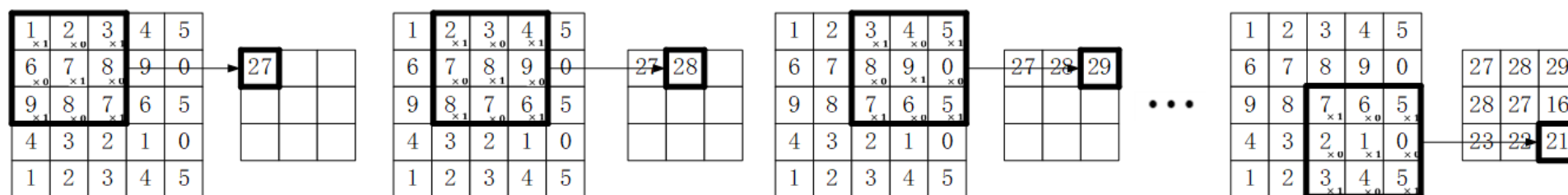
卷积操作示意

$$c_{11} = \sum_{u=1}^3 \sum_{v=1}^3 a_{uv} b_{uv} = a_{11}b_{11} + a_{12}b_{12} + \cdots + a_{33}b_{33} = 27$$

● 卷积神经网络概述

在深度学习中，上述过程中的矩阵 **A** 通常对应样本数据矩阵，矩阵 **B** 称之为卷积核，若使用同一个卷积核对整个数据矩阵进行卷积，则可得到原始数据矩阵的一个特征矩阵 **C**。

完成整个数据矩阵的卷积操作便可求得一个大小为 3×3 的特征矩阵 **C**：



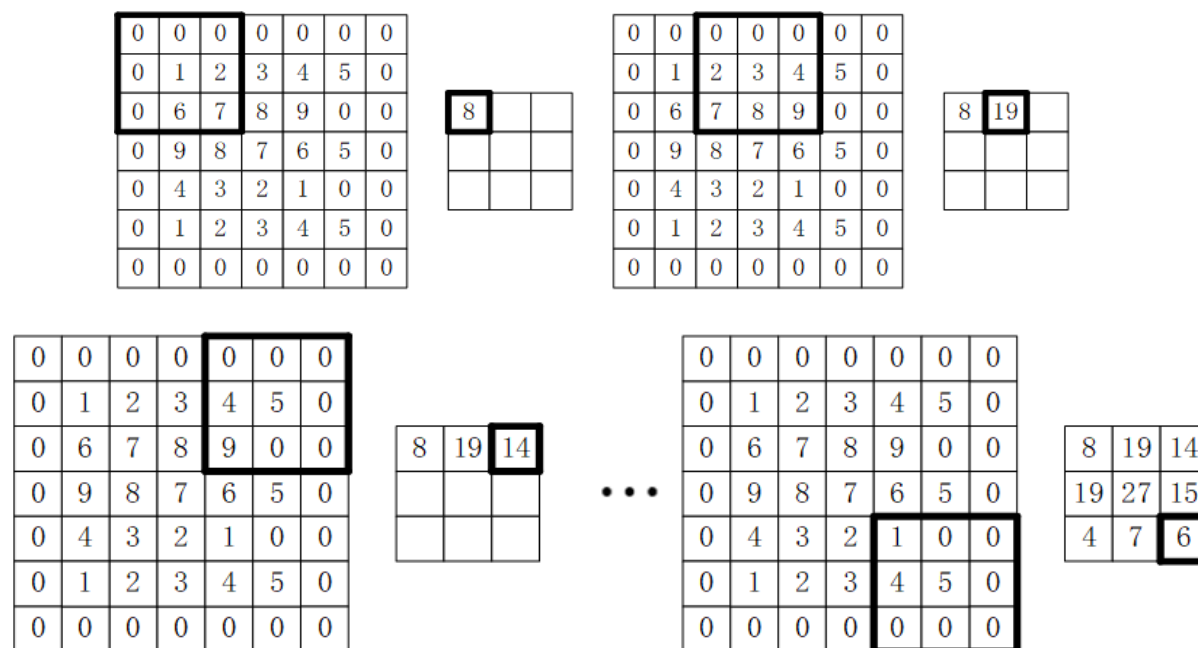
- 卷积神经网络概述

事实上，卷积核每次移动的单位可设定为不同长度，这一长度称之为步长（**stride**）。例如以上我们所讲卷积操作步长为1，若设置该过程中的卷积步长为2，则所得到的特征矩阵大小为 2×2 。

在实际操作中，步长越大，卷积操作所得到的特征矩阵越小，并且数据矩阵边界上的元素对特征矩阵的贡献较小，即丢失部分边界信息，为解决这些问题，通常会为原始数据填补上一圈或几圈元素，这一操作称之为填充（**padding**）

● 卷积神经网络概述

填充的圈数和填充的元素取值并无固定标准，但最常用的填充方式是为数据矩阵填充一圈或几圈**0**元素。如下图为填充后卷积操作：



- 卷积神经网络概述

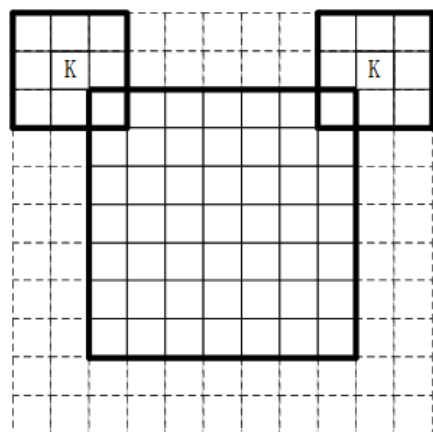
在实际操作中，数据矩阵和卷积核不一定为方阵，且水平方向和竖直方向的步长可设置为不同值。假设数据矩阵大小为 $M \times N$ ，卷积核大小为 $m \times n$ ，填充的圈数为 p ，水平方向和竖直方向的步长分别为 d_1, d_2 ，则有特征矩阵的行数和列数分别为：

$$R = \left\lfloor \frac{M + 2p - m}{d_1} \right\rfloor + 1 \quad C = \left\lfloor \frac{N + 2p - n}{d_2} \right\rfloor + 1$$

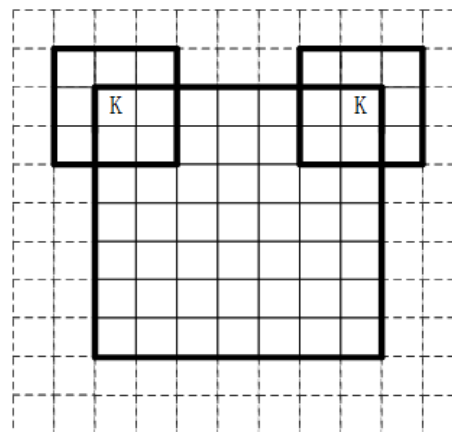
其中， $\lfloor \cdot \rfloor$ 表示向下取整。

● 卷积神经网络概述

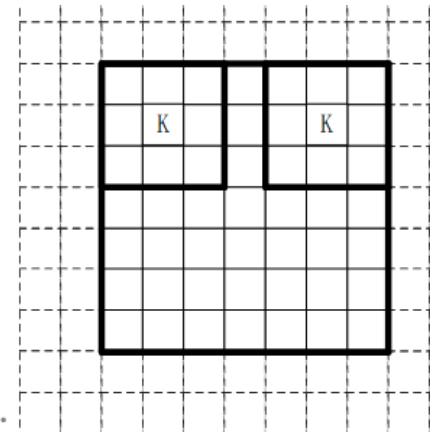
常见的填充策略：



(a) full卷积



(b) same卷积



(c) valid卷积

- 卷积神经网络概述

二、池化：

池化层亦称**Pooling**层，其操作是池化，即下采样，主要作用是通过去除输入的特征图中不重要的信息，使特征图变小，进行特征压缩，进一步减少参数量，且同时提取其中的有效信息。

● 卷积神经网络概述

池化的具体操作是定义池化窗口的大小。通常情况下，从某个池化窗口内进行采样的规则主要有取最大值、取最小值和取平均值三种，所对应的池化操作分别称之为最大池化、最小池化和均值池化。

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

6	8
3	4

1	2
1	0

3.25	5.25
2	2

(a) 数据矩阵 (b) 最大池化 (c) 最小池化 (d) 均值池化

● 卷积神经网络概述

三、激活函数：

对于如下图的数据矩阵、卷积核和特征矩阵，由卷积操作的定义可知特征矩阵中的各元素取值分别为：

a_1	a_2	a_3
a_4	a_5	a_6
a_7	a_8	a_9

数据矩阵

w_1	w_2
w_3	w_4

卷积核

y_1	y_2
y_3	y_4

特征矩阵

$$y_1 = \sigma(w_1 a_1 + w_2 a_2 + w_3 a_4 + w_4 a_5)$$

$$y_2 = \sigma(w_1 a_2 + w_2 a_3 + w_3 a_5 + w_4 a_6)$$

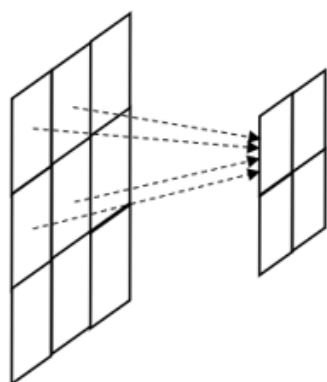
$$y_3 = \sigma(w_1 a_4 + w_2 a_5 + w_3 a_7 + w_4 a_8)$$

$$y_4 = \sigma(w_1 a_5 + w_2 a_6 + w_3 a_8 + w_4 a_9)$$

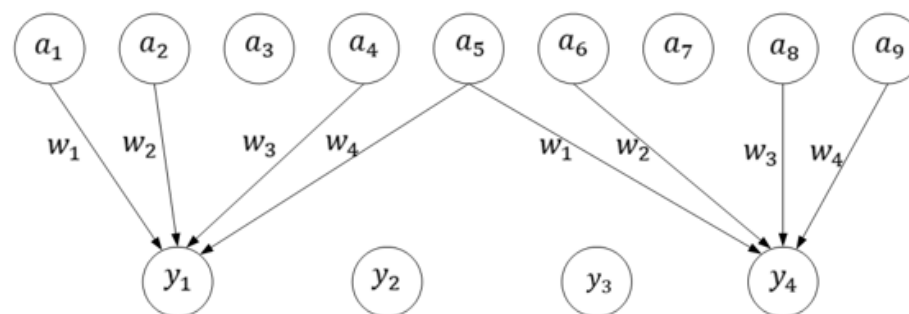
其中， σ 即为激活函数

● 卷积神经网络概述

由上式可得到特征矩阵中部分元素与数据矩阵元素相关性关系如下左图所示。更直观的，将左图中数据矩阵与特征矩阵展开为右图中神经元形式：



元素 y_1 相关性关系图



展开的元素 y_1, y_4 相关性关系图

不难发现采用卷积操作可使得两层神经元之间采用局部连接方式，即并非所有神经元都具有连接边，并且这些连接边共享一组参数，这使得卷积神经网络的参数数量较少，因此卷积神经网络可有效避免过拟合现象。

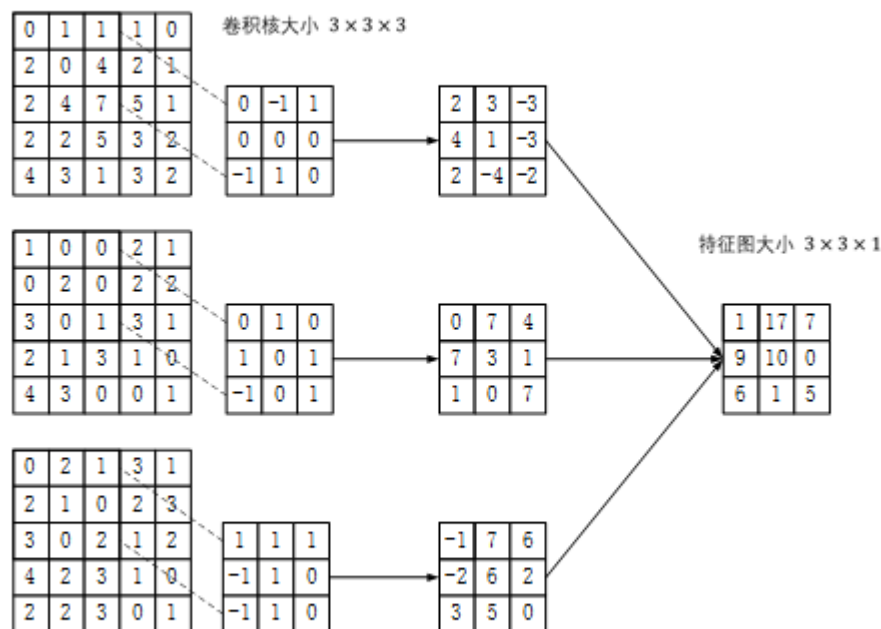
- 卷积神经网络概述

在实际应用当中，输入的数据矩阵可能并非简单的二维矩阵，例如采用RGB特征方式的彩色图像是由三个不同色彩通道的二维图堆叠形成的三维图像，因此可将其看成是由三个相同大小的数据矩阵堆叠而成的三维矩阵数据，对于此类数据，通常采用多通道卷积方式对其进行处理。

值得注意的是在卷积神经网络当中，卷积层的卷积核个数通常大于1，同层当中卷积核的个数也称为通道数，通常情况下卷积神经网络使用一组卷积核处理一个通道的数据矩阵。

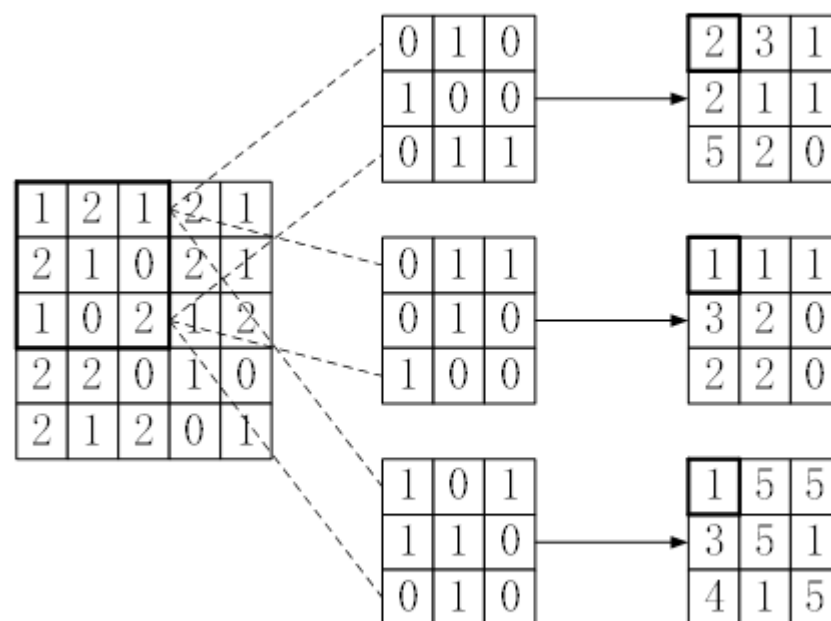
● 卷积神经网络概述

图片大小 $5 \times 5 \times 3$



图片大小 5×5

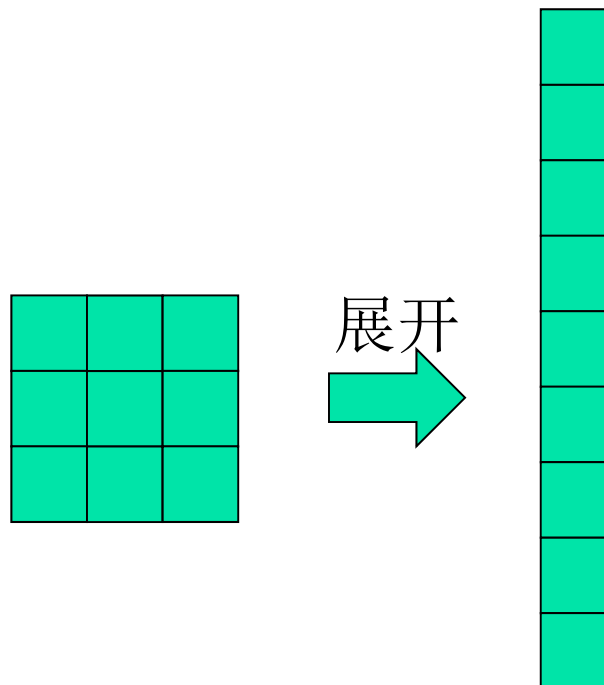
卷积核大小 3×3 特征图大小 3×3

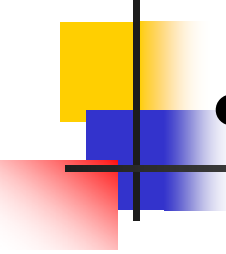


● 卷积神经网络概述

一个典型的卷积神经网络架构中包含多个交替的卷积层和池化层，而在网络的最后通常为多个全连接层。

但是显然，直接使用全连接层将特征矩阵展开会破坏空间结构信息，为解决这一问题，人们开始考虑使用卷积层来代替全连接层，从而使得整个网络中不包含全连接层。





● 卷积神经网络概述

使用卷积层来代替全连接层的具体操作如下：

- ① 假设全连接层的前一层输出为 $h \times w$ 大小的 d 张特征图，则可使用包含 d 个 $h \times w$ 大小卷积核所组成的卷积层代替该全连接层；
- ② 假设全连接层的前一层输出为 n 维向量，则可使用 n 个 1×1 大小卷积核所组成的卷积层代替该全连接层。若卷积神经网络中不包含全连接层，这称之为全卷积神经网络。

- 卷积神经网络概述

构造一个卷积神经网络的流程遵循构造神经网络的基本流程，也包括数据准备与预处理、模型初始化、确定优化目标、模型优化求解和验证模型性能等五个步骤，但由于卷积神经网络的结构与传统神经网络存在较大差异，并且所处理的数据形式较为特别，因此构造卷积神经网络的具体过程与构造传统神经网络的过程存在一定差异。

- 卷积神经网络概述

- 一. 数据准备与预处理
- 二. 模型初始化
- 三. 确定目标函数
- 四. 模型优化求解
- 五. 验证模型性能

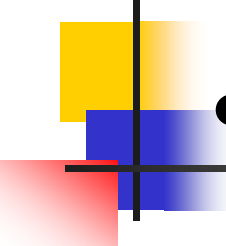
- 卷积神经网络概述

一. 数据准备与预处理

对于给定的机器学习任务，首先需要根据具体任务需求收集样本，卷积神经网络通常采用监督学习方法进行训练，因此对于所收集到的样本，还需对其进行标注从而得到数据集 D 。在机器学习中，构造一个模型不仅仅是指完成其训练过程，并且还需要对模型进行性能测试，因此需将数据集 D 划分为训练集 S 和测试集 T 。

- 卷积神经网络概述

卷积神经网络通常应用于计算机视觉任务，此类任务中所要处理的数据通常为图像数据，很多卷积神经网络对输入图像大小有所限制，而收集所得到的样本通常大小不一，为方便后续处理，需对样本进行**尺寸归一化（resize）处理**，即通过放缩将图像调整为满足要求的相同尺寸。



● 卷积神经网络概述

由于在数据准备阶段需要对数据进行人工标注，这导致很难得到大规模数据集，可在数据预处理阶段完成**样本增强**。

对于图像数据的样本增强较为方便，通过**放缩、裁剪、旋转、翻转**等简单操作便可实现有效的数据增强，而针对彩色图像，可采用**颜色空间转换或色彩扰动**方式实现样本增强。除上述对原始图像样本进行简单变换从而实现样本增强之外，还可使用深度置信网络、生成对抗网络等生成式模型生成**虚拟样本**从而实现样本增强。

- 卷积神经网络概述

二. 模型初始化

在卷积神经网络的模型初始化过程中，首先需要根据任务需求构建合适的网络模型，在卷积神经网络当中，在构建网络模型时便确定了很多超参数，例如卷积核的数目、大小等。

在确定了网络结构之后，还需确定模型的初始参数取值，与神经网络的模型初始化过程类似，卷积神经网络的可学习参数通常也使用随机初始化方式确定。

- 卷积神经网络概述

三. 确定目标函数

卷积神经网络通常采用监督学习方法进行训练，在确定了训练数据集 S 和初始模型之后，便可根据实际任务需求构造用于模型优化的目标函数。

如前所述，神经网络模型通常采用梯度下降法进行优化求解，因此其目标函数通常定义为模型在整个训练样本集上整体损失，但在深度学习中，训练样本集规模通常较大，若直接使用梯度下降算法则会导致参数更新过程中梯度计算与所有训练样本均相关，此时会导致模型参数更新缓慢，容易导致模型陷入局部最优，因此在构造卷积神经网络时通常采用小批量随机梯度下降算法。

 ● 卷积神经网络概述

小批量随机梯度下降算法每次训练时首先将训练样本集 S 打乱得到训练集 S' ，然后将 S' 等分为多个小的子集 $S'_1, S'_2, \dots, S'_k$ ，每个子集称为一个小批次（epoch），每个小批次中所包含的样本数称之为批大小（batchsize），其思想是每次迭代使用batch_size个样本来对参数进行更新。

假设 S'_i 批大小为 s ，则目标函数的具体形式为：

$$J'(\mathbf{W}, \mathbf{b}) = \frac{1}{s} \sum_{X_j \in S'_i} L(f(X_j), y_j)$$

其中 $f(X_j)$ 表示卷积神经网络对于样本 X_j 的输出， L 为损失函数，如前所述，对于分类问题常采用交叉熵损失函数等，而对于回归问题常采用平方误差损失函数或KL散度。

- 卷积神经网络概述

交叉熵损失函数 $J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$

- y 表示真实标记的分布
- $h(x)$ 为训练后模型的预测分布
- 交叉熵损失函数可以衡量 y 与 $h(x)$ 的相似性。即两个分布的交叉熵越小越相似

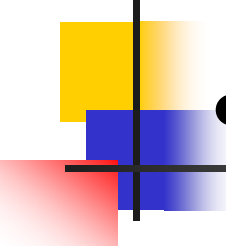
平方误差损失函数 $J(\theta) = \frac{\sum_{i=1}^n (y^{(i)} - y^{(i)'})^2}{n}$

 ● 卷积神经网络概述

为避免出现严重的过拟合现象，还可在损失函数中添加正则化项从而约束参数的取值。以采用交叉熵损失函数并添加 L^2 范数惩罚为例，卷积神经网络的每次参数更新所使用的目标函数可表示为：

$$J'(\mathbf{W}, \mathbf{b}) = -\frac{1}{s} \sum_{X_j \in S'_i} \sum_{t=1}^p y_{jt} \log f_t(X_j) + \frac{\alpha}{2} \|\mathbf{W}\|_2^2$$

其中 y_{jt} 为 X_j 所对应标签采用独热式编码所得到的 p 维向量 $\mathbf{y}_j$ 中的第 t 个元素取值， α 为正则化系数。



● 卷积神经网络概述

小批量随机梯度下降算法优点：

- 通过矩阵运算，每次在一个**batch**上优化神经网络参数并不会比单个数据慢太多。
- 每次使用一个**batch**可以大大减小收敛所需要的迭代次数，同时可以使收敛到的结果更加接近梯度下降的效果。
- 可实现并行化。

缺点：

- **batch_size**的不当选择可能会带来一些问题
- 盲目增大**batch_size**的坏处：
 - ① 内存利用率提高了，但是内存容量可能撑不住了。
 - ② 跑完一次 **epoch**（全数据集）所需的迭代次数减少，要想达到相同的精度，其所花费的时间大大增加了，从而对参数的修正也就显得更加缓慢。
 - ③ **Batch_Size** 增大到一定程度，其确定的下降方向已经基本不再变化。

- CNTK+Keras深度编程环境搭建

Windows下安装

- Anaconda

Anaconda3-4.3.1-Windows-x86_64-下载地址

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/>

https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-4.3.1-Windows-x86_64.exe

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-4.3.1-Windows-x86.exe>

- 创建工作目录和虚拟环境

- 安装CNTK（The Microsoft Cognitive Toolkit是深度学习计算环境）

<https://docs.microsoft.com/en-us/cognitive-toolkit/setup-cntk-on-your-machine>

- 安装Keras（深度学习建模环境）

<https://keras.io/>

- 编程环境：jupyter notebook

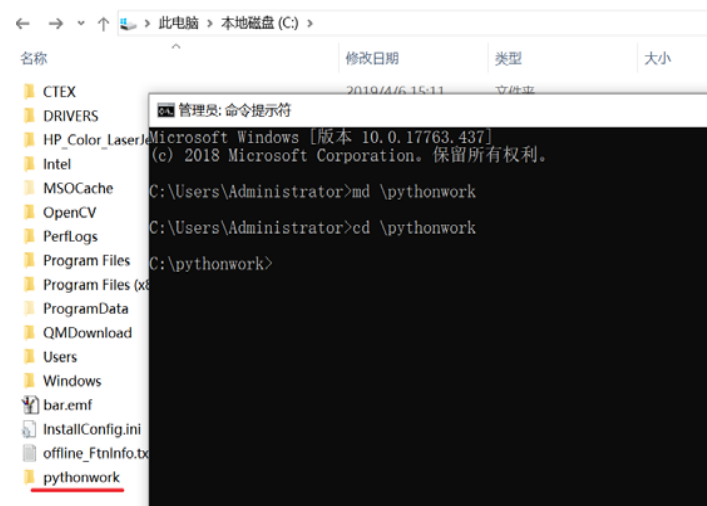
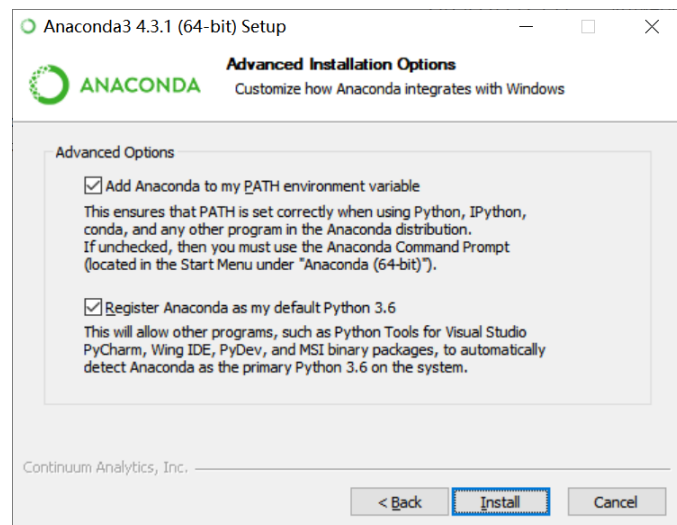
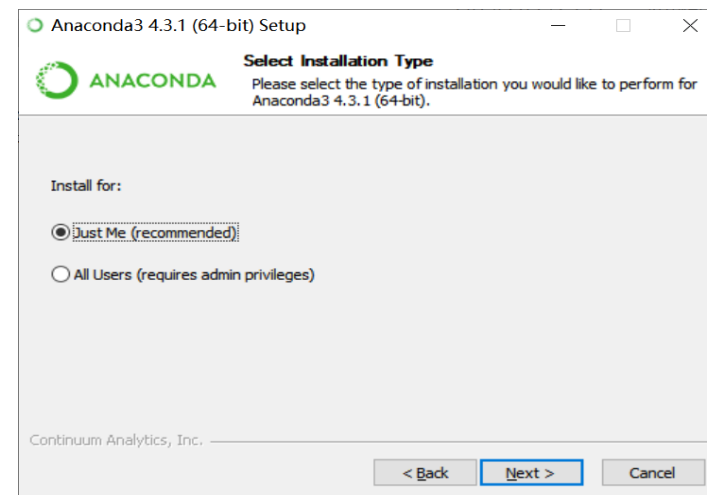
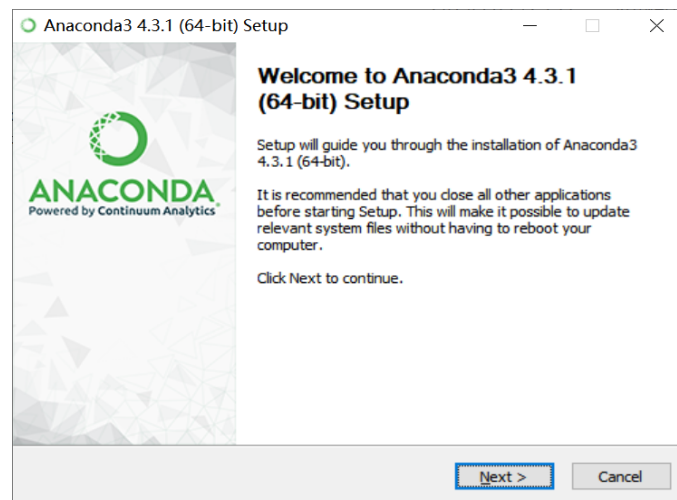
Anaconda3

We have been testing CNTK with Anaconda3 4.1.1 (64-bit) and Python versions 2.7 and 3.5, as well as Anaconda3 4.3.1 with Python version 3.6. If you do not have a Anaconda3 Python installation, install [Anaconda3 4.1.1 Python for Windows \(64-bit\)](#).

Below we assume Anaconda is installed and that it is listed before any other Python installations in your PATH. If you plan on using a GPU-enabled version of CNTK, you will need a CUDA 9 compliant graphics card and up-to-date graphics drivers installed on your system. Make sure you are installing CUDA 9.0 and not CUDA 9.1.

● CNTK+Keras深度编程环境搭建

Anaconda安装



● CNTK+Keras深度编程环境搭建

在工作目录下创建虚拟环境

`conda create --name cntkKerasPy36 python=3.6 anaconda`

```
管理员: 命令提示符 - conda create --name cntkKerasPy36 python=3.6 anaconda
Microsoft Windows [版本 10.0.17763.437]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>md \pythonwork

C:\Users\Administrator>cd \pythonwork

C:\pythonwork>conda create --name cntkKerasPy36 python=3.6 anaconda
Fetching package metadata .....
Solving package specifications: .

Package plan for installation in environment C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36:

The following NEW packages will be INSTALLED:

 _license:                1.1-py36_1
 alabaster:                0.7.10-py36_0
 anaconda:                 4.4.0-np112py36_0
 anaconda-client:          1.6.3-py36_0
 anaconda-navigator:       1.6.2-py36_0
 anaconda-project:         0.6.0-py36_0
 asn1crypto:               0.22.0-py36_0
 astroid:                  1.4.9-py36_0
 astropy:                  1.3.2-np112py36_0
 babel:                    2.4.0-py36_0
 backports:                1.0-py36_0
 beautifulsoup4:           4.6.0-py36_0
 bitarray:                 0.8.1-py36_1
 blas:                     1.0-mkl
 blaze:                     0.10.1-py36_0
```

● CNTK+Keras深度编程环境搭建

在虚拟工作环境下安装计算环境cntk

```
管理员: 命令提示符
Microsoft Windows [版本 10.0.17763.437]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>cd pythonwork
系统找不到指定的路径。

C:\Users\Administrator>cd \pythonwork

C:\pythonwork>activate cntkKerasPy36

(cntkKerasPy36) C:\pythonwork>pip install cntk
Collecting cntk
  Downloading https://files.pythonhosted.org/packages/b1/d3/0264ff331919441a225a64d0a59ac088eacc76c5825d0473f616e065a7b2/cntk-2.7.post1-cp36-cp36m-win_amd64.whl (64.0MB)
    100% |#####| 64.0MB 25kB/s
Requirement already satisfied: numpy>=1.11 in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from cntk)
Requirement already satisfied: scipy>=0.17 in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from cntk)
Installing collected packages: cntk
Successfully installed cntk-2.7
You are using pip version 9.0.1, however version 19.3.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

(cntkKerasPy36) C:\pythonwork>
```

● CNTK+Keras深度编程环境搭建

在虚拟工作环境下安装建模环境keras

```
C:\ 管理员: 命令提示符

(cntkKerasPy36) C:\pythonwork>pip install keras
Collecting keras
  Downloading https://files.pythonhosted.org/packages/ad/fd/6bfe87920d7f4fd475acd28500a42482b6b84479832bdc0fe9e589a60ceb/Keras-2.3.1-py2.py3-none-any.whl (377kB)
    100% |#####| 378kB 3.3MB/s
Collecting keras-applications>=1.0.6 (from keras)
  Downloading https://files.pythonhosted.org/packages/71/e3/19762fdcf62877ae9102edf6342d71b28fbfd9dea3d2f96a882ce099b03f/Keras_Applications-1.0.8-py3-none-any.whl (50kB)
    100% |#####| 51kB 3.4MB/s
Requirement already satisfied: h5py in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from keras)
Requirement already satisfied: pyyaml in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from keras)
Collecting keras-preprocessing>=1.0.5 (from keras)
  Downloading https://files.pythonhosted.org/packages/28/6a/8c1f62c37212d9fc441a7e26736df51ce6f0e38455816445471f10da4f0a/Keras_Preprocessing-1.1.0-py2.py3-none-any.whl (41kB)
    100% |#####| 51kB ...
Requirement already satisfied: numpy>=1.9.1 in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from keras)
Requirement already satisfied: scipy>=0.14 in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from keras)
Requirement already satisfied: six>=1.9.0 in c:\users\administrator\anaconda3\envs\cntkkeraspy36\lib\site-packages (from keras)
Installing collected packages: keras-applications, keras-preprocessing, keras
Successfully installed keras-2.3.1 keras-applications-1.0.8 keras-preprocessing-1.1.0
You are using pip version 9.0.1, however version 19.3.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

(cntkKerasPy36) C:\pythonwork>
```

● CNTK+Keras深度编程环境搭建

jupyter notebook——导入cntk遇到问题

```
管理员: 命令提示符 - python
(cntkKerasPy36) C:\pythonwork>python
Python 3.6.1 |Anaconda 4.4.0 (64-bit)| (default, May 11 2017, 13:25:24) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import cntk
C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\cntk\cntk_py_init.py:84: UserWarning:

##### Missing optional dependency (GPU-Specific) #####
CNTK may crash if the component that depends on those dependencies is loaded.
Visit https://docs.microsoft.com/en-us/cognitive-toolkit/Setup-Windows-Python#optional-gpu-specific-packages for more information.
#####
If you intend to use CNTK without GPU support, you can ignore the (likely) GPU-specific warning!
#####

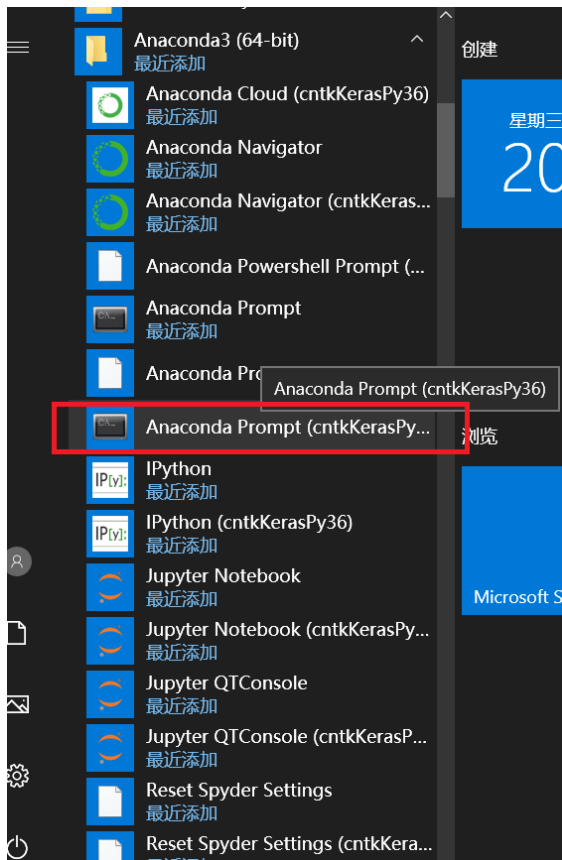
warnings.warn(WARNING_MSG_GPU_ONLY % ('GPU-Specific', 'https://docs.microsoft.com/en-us/cognitive-toolkit/Setup-Windows-Python#optional-gpu-specific-packages'))
Traceback (most recent call last):
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy\core\__init__.py", line 16, in <module>
    from . import multiarray
ImportError: DLL load failed: 操作系统无法运行 %1。

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\cntk\_init_.py", line 25, in <module>
    import numpy as np
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy\_init_.py", line 142, in <module>
    from . import add_newdocs
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy\add_newdocs.py", line 13, in <module>
    from numpy.lib import add_newdoc
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy\lib\_init_.py", line 8, in <module>
    from .type_check import *
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy\lib\type_check.py", line 11, in <module>
    import numpy.core.numeric as _nx
  File "C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy\core\__init__.py", line 24, in <module>
    raise ImportError(msg)
ImportError:
Importing the multiarray numpy extension module failed. Most
likely you are trying to import a failed build of numpy.
If you're working with a numpy git repo, try `git clean -xdf` (removes all
files not under version control). Otherwise reinstall numpy.
```

● CNTK+Keras深度编程环境搭建

jupyter notebook——导入cntk遇到问题的解决方案



```
管理员: Anaconda Prompt (cntkKerasPy36)

(C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36) C:\Users\Administrator>pip uninstall numpy
Uninstalling numpy-1.12.1:
  Would remove:
    c:\users\administrator\anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy
    c:\users\administrator\anaconda3\envs\cntkKerasPy36\lib\site-packages\numpy-1.12.1-py3.6.egg-info
Proceed (y/n)? y
  Successfully uninstalled numpy-1.12.1

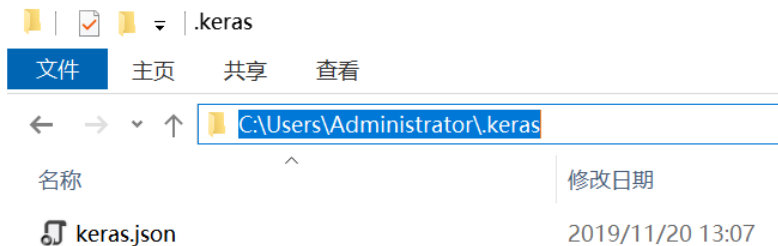
(C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36) C:\Users\Administrator>pip install numpy
Collecting numpy
  Using cached https://files.pythonhosted.org/packages/b0/ee/5ff445dd43b9820e5494d21240e689d3b7cb52bc93f4f164eba84206cd0d/numpy-1.17.4-cp36-cp36m-win_amd64.whl
Installing collected packages: numpy
Successfully installed numpy-1.17.4

(C:\Users\Administrator\Anaconda3\envs\cntkKerasPy36) C:\Users\Administrator>
```

● CNTK+Keras深度编程环境搭建

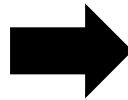
配置Keras后端

Windows下配置文件路径C:\Users\Administrator\.keras



keras.json - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
{  
  "floatx": "float32",  
  "epsilon": 1e-07,  
  "backend": "tensorflow",  
  "image_data_format": "channels_last"  
}
```



keras.json - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
{  
  "floatx": "float32",  
  "epsilon": 1e-07,  
  "backend": "cntk",  
  "image_data_format": "channels_last"  
}
```

- CIFAR图像分类网络学习

- CIFAR数据集下载及浏览
- 基于CNN的CIFAR图像分类实践

管理员: 命令提示符 - jupyter notebook

Microsoft Windows [版本 10.0.17763.437]

(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>cd \pythonwork

C:\pythonwork>activate cntkKerasPy36

(cntkKerasPy36) C:\pythonwork>jupyter notebook

[I 12:39:57.537 NotebookApp] Serving notebooks from local directory: C:\pythonwork

[I 12:39:57.552 NotebookApp] 0 active kernels

[I 12:39:57.554 NotebookApp] The Jupyter Notebook is running at: http://localhost:8888/?token=24541652e477e1911e949a0666d1eb7ca10d3dc41581eab1

[I 12:39:57.554 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).

[C 12:39:57.556 NotebookApp]

- 打开命令窗口
- 进入工作目录
- 激活虚拟运行环境
- 打开jupyter notebook

● CIFAR图像分类网络学习

● CIFAR数据集下载及浏览

导入Keras及cifar10相关模块

```
In [3]: import numpy
        from keras.datasets import cifar10      //导入cifar10模块
        import numpy as np                    //矩阵运算
        np.random.seed(10)                   //设置生产随机数的seed
```

数据集下载并显示相关信息

```
In [4]: (x_img_train, y_label_train), \
        (x_img_test, y_label_test) = cifar10.load_data() //读取数据，提前把数据集下载好，这里就不会自动下载
                                                //Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
```

```
In [5]: print('train:', len(x_img_train))
        print('test :', len(x_img_test))              //打印数据项数

train: 50000
test : 10000
```

```
In [6]: x_img_train.shape                          //查看训练数据的项数，图片的维数和通道数
```

```
Out[6]: (50000, 32, 32, 3)
```

```
In [7]: y_label_train.shape                       //查看训练标签的项数
```

```
Out[7]: (50000, 1)
```

```
In [8]: x_img_test.shape                         //查看测试数据的项数，图片的维数和通道数
```

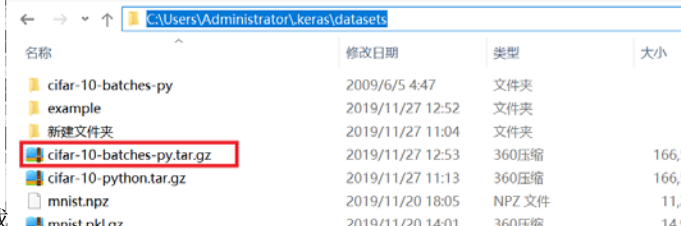
```
Out[8]: (10000, 32, 32, 3)
```

```
In [7]: x_img_test[0]                           //查看图片每一个像素的像素值，是一个RGB向量
```

```
Out[7]: array([[158, 112,  49],
               [159, 111,  47],
               [165, 116,  51],
               ...,
               [137,  95,  36]])
```

//cifar10数据集包括10个类别:

0:airplane
1:aotomobile
2:bird
3:cat
4:deer
5:dog
6:frog
7:horse
8:ship
9:truck



名称	修改日期	类型	大小
cifar-10-batches-py	2009/6/5 4:47	文件夹	
example	2019/11/27 12:52	文件夹	
新建文件夹	2019/11/27 11:04	文件夹	
cifar-10-batches-py.tar.gz	2019/11/27 12:53	360压缩	166,
cifar-10-python.tar.gz	2019/11/27 11:13	360压缩	166,
mnist.npz	2019/11/20 18:05	NPZ 文件	11,
mnist.pkl.gz	2019/11/20 14:01	360压缩	14,

● CIFAR图像分类网络学习

```
In [9]: y_label_test.shape
```

```
Out[9]: (10000, 1)
```

//查看测试标签的项数

```
In [10]: label_dict={0:"airplane",1:"automobile",2:"bird",3:"cat",4:"deer",
                    5:"dog",6:"frog",7:"horse",8:"ship",9:"truck"}
```

```
In [11]: import matplotlib.pyplot as plt
def plot_images_labels_prediction(images, labels, prediction,
                                  idx, num=10):
    fig = plt.gcf()
    fig.set_size_inches(12, 14)
    if num>25: num=25
    for i in range(0, num):
        ax=plt.subplot(5, 5, 1+i)
        ax.imshow(images[idx], cmap='binary')

        title=str(i)+' '+label_dict[labels[i][0]]
        if len(prediction)>0:
            title+=' => '+label_dict[prediction[i]]

        ax.set_title(title, fontsize=10)
        ax.set_xticks([]); ax.set_yticks([])
        idx+=1
    plt.show()
```

//编写函数输出训练数据前10项

```
In [12]: plot_images_labels_prediction(x_img_train, y_label_train, [], 0)
```



● CIFAR图像分类网络学习

```
In [13]: print('x_img_test:', x_img_test.shape)
         print('y_label_test :', y_label_test.shape)
```

//输出测试图及标签相关信息

```
x_img_test: (10000, 32, 32, 3)
y_label_test : (10000, 1)
```

Image normalize

```
In [14]: x_img_train[0][0][0]
```

//查看训练数据第1个图像的第1个点

```
Out[14]: array([59, 62, 63], dtype=uint8)
```

```
In [15]: x_img_train_normalize = x_img_train.astype('float32') / 255.0
         x_img_test_normalize = x_img_test.astype('float32') / 255.0
```

//将图像数据归一化可以提升模型的准确率

```
In [16]: x_img_train_normalize[0][0][0]
```

//查看归一化后的结果

```
Out[16]: array([0.23137255, 0.24313726, 0.24705882], dtype=float32)
```

● CIFAR图像分类网络学习

//进行One-Hot Encoding的目的：希望预测图像的类型，例如“船”的图像的Label是8，经过一位有效编码（One-Hot Encoding）转换为0000000010，10个数字正好对应输出层10个神经元

对label进行预处理

```
In [17]: y_label_train.shape
```

//训练数据标签50000项，每一项是0-9之间数字

```
Out[17]: (50000, 1)
```

```
In [18]: y_label_train[:5]
```

//查看标签前5项数据

```
Out[18]: array([[6],
                [9],
                [9],
                [4],
                [1]], dtype=uint8)
```

```
In [19]: from keras.utils import np_utils
y_label_train_OneHot = np_utils.to_categorical(y_label_train)
y_label_test_OneHot = np_utils.to_categorical(y_label_test)
```

//利用Keras提供的np_utils.to_categorical方法对标签进行One-Hot Encoding转换

```
In [20]: y_label_train_OneHot.shape
```

//查看转换后数据的项目，以及每一项的维数

```
Out[20]: (50000, 10)
```

```
In [21]: y_label_train_OneHot[:5]
```

//打印转换后的输出结果

```
Out[21]: array([[0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
                [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1.],
                [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1.],
                [0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
                [0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.]], dtype=float32)
```

● CIFAR图像分类网络学习

● 基于CNN的CIFAR图像分类实践

导入Keras及cifar10相关模块

```
In [1]: from keras.datasets import cifar10
import numpy as np
np.random.seed(10)
```

//导入相关模块

```
In [2]: (x_img_train,y_label_train),(x_img_test,y_label_test)=cifar10.load_data()
```

//读入cifar10数据

```
In [3]: print("train data:", 'images:', x_img_train.shape,
          " labels:", y_label_train.shape)
print("test data:", 'images:', x_img_test.shape ,
      " labels:", y_label_test.shape)
```

//显示训练和测试数据的相关信息

```
train data: images: (50000, 32, 32, 3) labels: (50000, 1)
test data: images: (10000, 32, 32, 3) labels: (10000, 1)
```

```
In [4]: x_img_train_normalize = x_img_train.astype('float32') / 255.0
x_img_test_normalize = x_img_test.astype('float32') / 255.0
```

//数据归一化，提升模型预测准确率和收敛速度

```
In [5]: from keras.utils import np_utils
y_label_train_OneHot = np_utils.to_categorical(y_label_train)
y_label_test_OneHot = np_utils.to_categorical(y_label_test)
```

//对标签进行一位有效编码转换

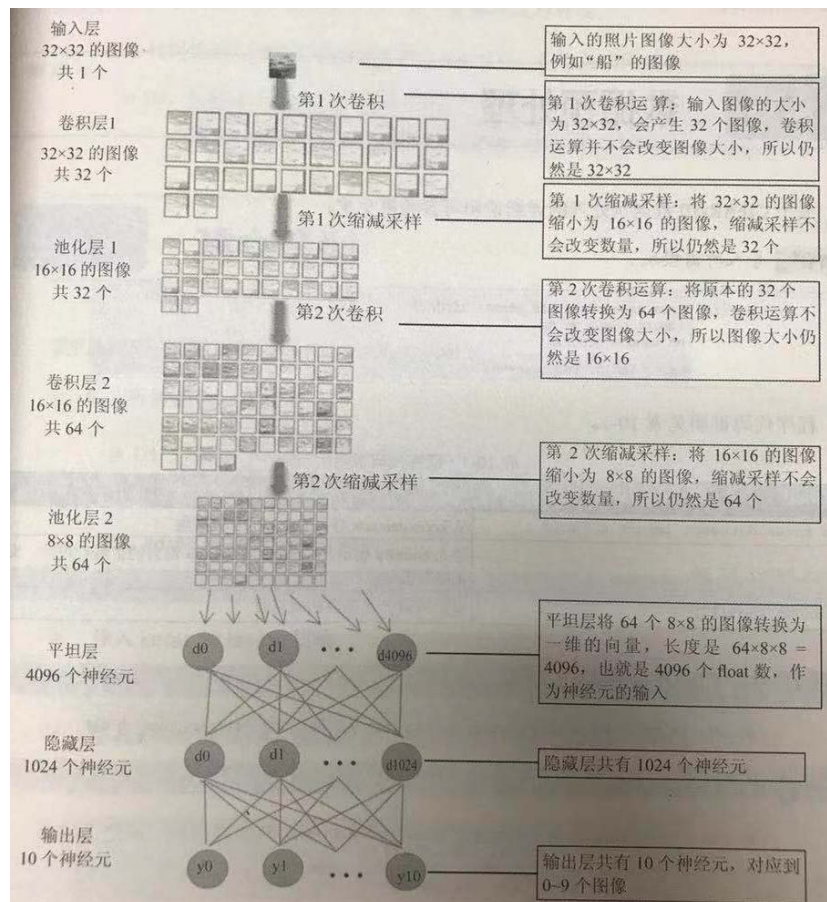
```
In [6]: y_label_test_OneHot.shape
```

//输出编码后的结果

```
Out[6]: (10000, 10)
```

● CIFAR图像分类网络学习

模型建立



```
In [7]: from keras.models import Sequential
        from keras.layers import Dense, Dropout, Activation, Flatten
        from keras.layers import Conv2D, MaxPooling2D, ZeroPadding2D

In [8]: model = Sequential()                                //建立Sequential线性堆叠模型

In [9]: #卷积层1

In [10]: model.add(Conv2D(filters=32, kernel_size=(3, 3),      //第1层卷积，设置32个滤波器，核大小3*3
                           input_shape=(32, 32, 3),           //输入图片维度32*32*3
                           activation='relu',                  //设置ReLU激活函数
                           padding='same'))                    //让卷积运算并不会改变图像大小

In [11]: model.add(Dropout(rate=0.25))                       //每次迭代训练时随机放弃25%神经元，避免过度拟合

In [12]: model.add(MaxPooling2D(pool_size=(2, 2)))           //第1次下采样，不改变特征数量

In [13]: #卷积层2与池化层2

In [14]: model.add(Conv2D(filters=64, kernel_size=(3, 3),      //第2层卷积，设置64个滤波器，将32
                           activation='relu', padding='same'))  //个特征图像转换为64个特征图像

In [15]: model.add(Dropout(0.25))                             //每次迭代训练时随机放弃25%神经元，避免过度拟合

In [16]: model.add(MaxPooling2D(pool_size=(2, 2)))

In [17]: #Step3-建立神经网络(平坦层、隐藏层、输出层)

In [18]: model.add(Flatten())                                  //建立平坦层，将池化层2的64个8*8的图像转为1维向量，
        model.add(Dropout(rate=0.25))                          //即64*8*8=4096个浮点数，对应4096个神经元

In [19]: model.add(Dense(1024, activation='relu'))            //建立隐藏层，共有1024个神经元
        model.add(Dropout(rate=0.25))

In [20]: model.add(Dense(10, activation='softmax'))           //建立输出层，共有10个神经元，对应10个
                                                                类别

In [21]: print(model.summary())
```

● CIFAR图像分类网络学习

模型训练

```
In [22]: try:
          model.load_weights("SaveModel/cifarCnnModelnew1.h5")
          print("加载模型成功!继续训练模型")
        except :
          print("加载模型失败!开始训练一个新模型")
```

加载模型失败!开始训练一个新模型

//loss: 设置损失函数, 在深度学习中一般使用cross_entropy (交叉熵) 训练的效果比较好

//optimizer: 设置优化器为adam可以让训练更快收敛, 并提高准确率

//metrics: 设置评估模型的方式是准确率

```
In [23]: model.compile(loss='categorical_crossentropy', //采用compile方法对模型进行设置
                  optimizer='adam', metrics=['accuracy'])
```

```
In [24]: train_history=model.fit(x_img_train_normalize, y_label_train_OneHot, //采用model.fit开始训练, 训练过程存储在train_history中, 前两个参数为归
                  validation_split=0.2, //第3个参数表示训练前自动将80%作为训练数据, 20%作为验证数据
                  epochs=10, batch_size=128, verbose=1) //最后3个参数表示: 训练周期、每一批输入的数据个数、设置输出进度条
```

Train on 40000 samples, validate on 10000 samples

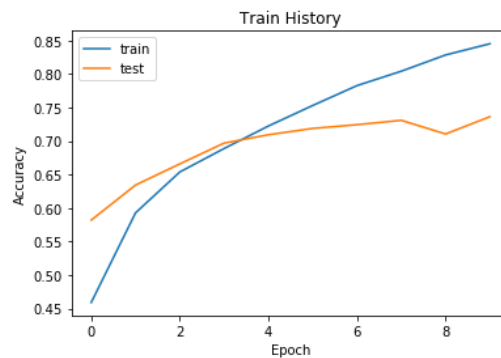
```
Epoch 1/10
40000/40000 [=====] - 116s - loss: 1.5030 - acc: 0.4591 - val_loss: 1.2820 - val_acc: 0.5820
Epoch 2/10
40000/40000 [=====] - 115s - loss: 1.1417 - acc: 0.5928 - val_loss: 1.1253 - val_acc: 0.6343
Epoch 3/10
40000/40000 [=====] - 116s - loss: 0.9868 - acc: 0.6538 - val_loss: 1.0072 - val_acc: 0.6659
Epoch 4/10
40000/40000 [=====] - 116s - loss: 0.8813 - acc: 0.6888 - val_loss: 0.9408 - val_acc: 0.6969
Epoch 5/10
40000/40000 [=====] - 118s - loss: 0.7901 - acc: 0.7224 - val_loss: 0.8733 - val_acc: 0.7094
Epoch 6/10
40000/40000 [=====] - 117s - loss: 0.7053 - acc: 0.7530 - val_loss: 0.8509 - val_acc: 0.7188
Epoch 7/10
40000/40000 [=====] - 117s - loss: 0.6258 - acc: 0.7827 - val_loss: 0.8187 - val_acc: 0.7244
Epoch 8/10
40000/40000 [=====] - 118s - loss: 0.5609 - acc: 0.8042 - val_loss: 0.7904 - val_acc: 0.7309
Epoch 9/10
40000/40000 [=====] - 120s - loss: 0.4956 - acc: 0.8284 - val_loss: 0.8333 - val_acc: 0.7105
Epoch 10/10
40000/40000 [=====] - 117s - loss: 0.4400 - acc: 0.8452 - val_loss: 0.7658 - val_acc: 0.7363
```

● CIFAR图像分类网络学习

画图输出准确率与误差

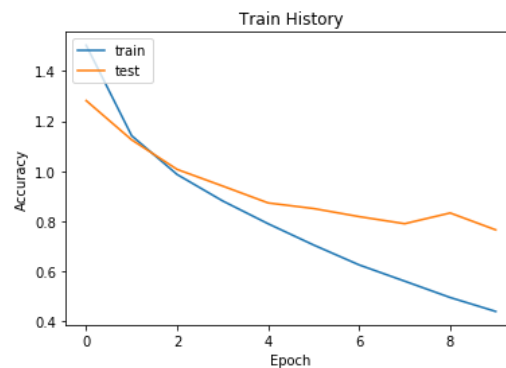
```
In [25]: import matplotlib.pyplot as plt
def show_train_history(train_acc, test_acc):
    plt.plot(train_history.history[train_acc])
    plt.plot(train_history.history[test_acc])
    plt.title('Train History')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend(['train', 'test'], loc='upper left')
    plt.show()
```

```
In [26]: show_train_history('acc', 'val_acc')
```



//蓝色是训练的准确率
//橙色是验证的准确率

```
In [27]: show_train_history('loss', 'val_loss')
```



//蓝色是训练的误差
//橙色是验证的误差

● CIFAR图像分类网络学习

模型验证与预测

```
In [28]: scores = model.evaluate(x_img_test_normalize,  
                                y_label_test_OneHot, verbose=0)  
scores[1]
```

//采用test测试数据集来评估模型的准确率

```
Out[28]: 0.7339
```

```
In [29]: prediction=model.predict_classes(x_img_test_normalize)  
9984/10000 [=====>.] - ETA: 0s
```

//模型预测

```
In [30]: prediction[:10]
```

//查看前10项的预测结果

```
Out[30]: array([3, 8, 8, 0, 6, 6, 1, 6, 3, 1], dtype=int64)
```


● CIFAR图像分类网络学习

图示预测结果

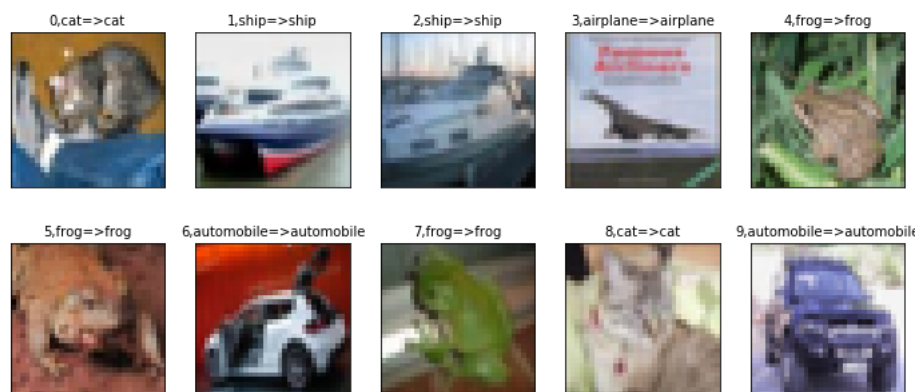
```
In [31]: label_dict={0:"airplane",1:"automobile",2:"bird",3:"cat",4:"deer",
                    5:"dog",6:"frog",7:"horse",8:"ship",9:"truck"}
```

```
In [32]: import matplotlib.pyplot as plt
def plot_images_labels_prediction(images, labels, prediction,
                                  idx, num=10):
    fig = plt.gcf()
    fig.set_size_inches(12, 14)
    if num>25: num=25
    for i in range(0, num):
        ax=plt.subplot(5, 5, 1+i)
        ax.imshow(images[idx], cmap='binary')

        title=str(i)+' '+label_dict[labels[i][0]]
        if len(prediction)>0:
            title+='>' +label_dict[prediction[i]]

        ax.set_title(title, fontsize=10)
        ax.set_xticks([]): ax.set_yticks([])
        idx+=1
    plt.show()
```

```
In [33]: plot_images_labels_prediction(x_img_test, y_label_test,
                                       prediction, 0, 10)
```



● CIFAR图像分类网络学习

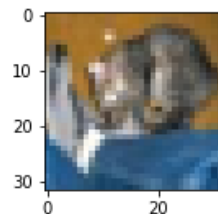
查看每一个图预测为不同类别的概率

```
In [34]: Predicted_Probability=model.predict(x_img_test_normalize)
```

```
In [35]: def show_Predicted_Probability(y, prediction,
                                         x_img, Predicted_Probability, i):
    print('label:', label_dict[y[i][0]],
          'predict:', label_dict[prediction[i]])
    plt.figure(figsize=(2, 2))
    plt.imshow(np.reshape(x_img_test[i], (32, 32, 3)))
    plt.show()
    for j in range(10):
        print(label_dict[j]+
              ' Probability:%1.9f'%(Predicted_Probability[i][j]))
```

```
In [36]: show_Predicted_Probability(y_label_test, prediction,
                                     x_img_test, Predicted_Probability, 0)
```

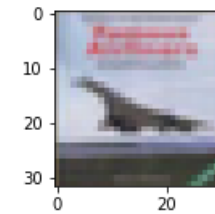
label: cat predict: cat



```
airplane Probability:0.007275202
automobile Probability:0.003484705
bird Probability:0.011789843
cat Probability:0.521885097
deer Probability:0.011080682
dog Probability:0.382938087
frog Probability:0.032875706
horse Probability:0.005801901
ship Probability:0.016262261
truck Probability:0.006606583
```

```
In [37]: show_Predicted_Probability(y_label_test, prediction,
                                     x_img_test, Predicted_Probability, 3)
```

label: airplane predict: airplane



```
airplane Probability:0.755786240
automobile Probability:0.002187771
bird Probability:0.016898982
cat Probability:0.001257468
deer Probability:0.004320458
dog Probability:0.000057983
frog Probability:0.000155273
horse Probability:0.000030577
ship Probability:0.217318073
truck Probability:0.001987258
```

● CIFAR图像分类网络学习

输出混淆矩阵

```
In [38]: prediction.shape
Out[38]: (10000, )

In [39]: y_label_test.shape
Out[39]: (10000, 1)

In [40]: y_label_test
Out[40]: array([[3],
               [8],
               [8],
               ...,
               [5],
               [1],
               [7]])

In [41]: y_label_test.reshape(-1)
Out[41]: array([3, 8, 8, ..., 5, 1, 7])

In [42]: import pandas as pd
print(label_dict)
pd.crosstab(y_label_test.reshape(-1), prediction,
            rownames=['label'], colnames=['predict'])

{0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer', 5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'}

Out[42]: predict    0    1    2    3    4    5    6    7    8    9
label
0    820    7    49    12    15     1    19     5    48    24
1     25   817    19    13     8     4    14     5    30    65
2     57     4   636    25    94    54    95    21     8     6
3     23     8    99   460    76   165   129    21     8    11
4     20     1    66    33   743    22    80    26     8     1
5     15     1    70   129    56   628    64    29     3     5
6      3     3    33    26    21    13   896     3     2     0
7     16     2    50    23    83    64    16   740     5     1
8     69    21    21    12    10     9    17     1   825    15
9     43    80    22    17     4    12    17    12    19   774

In [43]: print(label_dict)

{0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer', 5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'}
```

//真实label是3的test数据，被预测为label 5
的个数是165个

● CIFAR图像分类网络学习

保存结果

Save model to JSON

```
In [44]: model_json = model.to_json()
with open("SaveModel/cifarCnnModelnew.json", "w") as json_file:
    json_file.write(model_json)
```

Save Model to YAML

```
In [45]: model_yaml = model.to_yaml()
with open("SaveModel/cifarCnnModelnew.yaml", "w") as yaml_file:
    yaml_file.write(model_yaml)
```

Save Weight to h5

```
In [46]: model.save_weights("SaveModel/cifarCnnModelnew.h5")
print("Saved model to disk")
```

Saved model to disk

```
In [47]: model.save_weights("SaveModel/cifarCnnModelnew.h5")
print("Saved model to disk")
```

Saved model to disk

● CIFAR图像分类网络学习

建立3次卷积运算

In [9]: #卷积层1与池化层1

```
In [10]: model.add(Conv2D(filters=32, kernel_size=(3, 3), input_shape=(32, 32, 3),
activation='relu', padding='same'))
model.add(Dropout(0.3))
model.add(Conv2D(filters=32, kernel_size=(3, 3),
activation='relu', padding='same'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

//增加一次卷积运算

In [11]: #卷积层2与池化层2

```
In [12]: model.add(Conv2D(filters=64, kernel_size=(3, 3),
activation='relu', padding='same'))
model.add(Dropout(0.3))
model.add(Conv2D(filters=64, kernel_size=(3, 3),
activation='relu', padding='same'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

In [13]: #卷积层3与池化层3

```
In [14]: model.add(Conv2D(filters=128, kernel_size=(3, 3),
activation='relu', padding='same'))
model.add(Dropout(0.3))
model.add(Conv2D(filters=128, kernel_size=(3, 3),
activation='relu', padding='same'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

//增加卷积模块

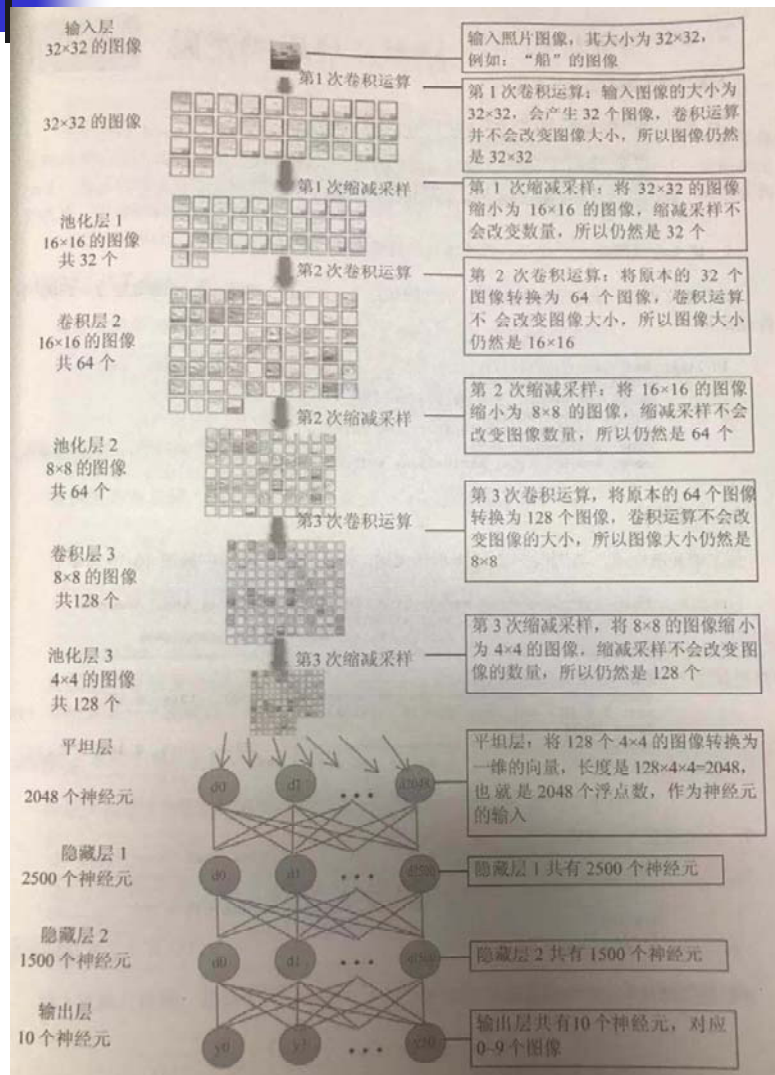
```
In [16]: model.add(Flatten())
model.add(Dropout(0.3))
model.add(Dense(2500, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1500, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(10, activation='softmax'))
```

//2048个神经元, $128*4*4=2048$

```
In [20]: train_history=model.fit(X_img_train_normalize, y_label_train_OneHot,
validation_split=0.2,
epochs=50, batch_size=300, verbose=1)
```

//增加训练次数

● CIFAR图像分类网络学习



```
In [7]: from keras.models import Sequential
        from keras.layers import Dense, Dropout, Activation, Flatten
        from keras.layers import Conv2D, MaxPooling2D, ZeroPadding2D

In [8]: model = Sequential()

In [9]: #卷积层1与池化层1

In [10]: model.add(Conv2D(filters=32, kernel_size=(3, 3), input_shape=(32, 32, 3),
        activation='relu', padding='same'))
        model.add(Dropout(0.3))
        model.add(Conv2D(filters=32, kernel_size=(3, 3),
        activation='relu', padding='same'))
        model.add(MaxPooling2D(pool_size=(2, 2)))

In [11]: #卷积层2与池化层2

In [12]: model.add(Conv2D(filters=64, kernel_size=(3, 3),
        activation='relu', padding='same'))
        model.add(Dropout(0.3))
        model.add(Conv2D(filters=64, kernel_size=(3, 3),
        activation='relu', padding='same'))
        model.add(MaxPooling2D(pool_size=(2, 2)))

In [13]: #卷积层3与池化层3

In [14]: model.add(Conv2D(filters=128, kernel_size=(3, 3),
        activation='relu', padding='same'))
        model.add(Dropout(0.3))
        model.add(Conv2D(filters=128, kernel_size=(3, 3),
        activation='relu', padding='same'))
        model.add(MaxPooling2D(pool_size=(2, 2)))

In [15]: #建立神经网络(平坦层、隐藏层、输出层)

In [16]: model.add(Flatten())
        model.add(Dropout(0.3))
        model.add(Dense(2500, activation='relu'))
        model.add(Dropout(0.3))
        model.add(Dense(1500, activation='relu'))
        model.add(Dropout(0.3))
        model.add(Dense(10, activation='softmax'))
```

● CIFAR图像分类网络学习

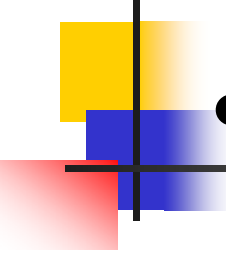
In [17]: `print(model.summary())`

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 32, 32, 32)	896
dropout_1 (Dropout)	(None, 32, 32, 32)	0
conv2d_2 (Conv2D)	(None, 32, 32, 32)	9248
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_3 (Conv2D)	(None, 16, 16, 64)	18496
dropout_2 (Dropout)	(None, 16, 16, 64)	0
conv2d_4 (Conv2D)	(None, 16, 16, 64)	36928
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_5 (Conv2D)	(None, 8, 8, 128)	73856
dropout_3 (Dropout)	(None, 8, 8, 128)	0
conv2d_6 (Conv2D)	(None, 8, 8, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten_1 (Flatten)	(None, 2048)	0
dropout_4 (Dropout)	(None, 2048)	0
dense_1 (Dense)	(None, 2500)	5122500
dropout_5 (Dropout)	(None, 2500)	0
dense_2 (Dense)	(None, 1500)	3751500
dropout_6 (Dropout)	(None, 1500)	0
dense_3 (Dense)	(None, 10)	15010
Total params: 9,176,018.0		
Trainable params: 9,176,018.0		
Non-trainable params: 0.0		
None		

In [18]: `model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])`

In [34]: `train_history=model.fit(X_img_train_normalize, y_label_train_OneHot, validation_split=0.2, epochs=50, batch_size=300, verbose=1)`

```
Epoch 45/50
40000/40000 [=====] - 345s 9ms/step - loss: 0.1488 - accuracy: 0.9972 - val_loss: 0.7493
- val_accuracy: 0.8007
Epoch 46/50
40000/40000 [=====] - 345s 9ms/step - loss: 0.1531 - accuracy: 0.9970 - val_loss: 0.8073
- val_accuracy: 0.7899
Epoch 47/50
40000/40000 [=====] - 345s 9ms/step - loss: 0.1435 - accuracy: 0.9976 - val_loss: 0.7640
- val_accuracy: 0.8013
Epoch 48/50
40000/40000 [=====] - 345s 9ms/step - loss: 0.1460 - accuracy: 0.9975 - val_loss: 0.7357
- val_accuracy: 0.8038
Epoch 49/50
40000/40000 [=====] - 345s 9ms/step - loss: 0.1409 - accuracy: 0.9979 - val_loss: 0.7595
- val_accuracy: 0.8044
Epoch 50/50
40000/40000 [=====] - 346s 9ms/step - loss: 0.1420 - accuracy: 0.9976 - val_loss: 0.7660
- val_accuracy: 0.8011
```



● 实验4

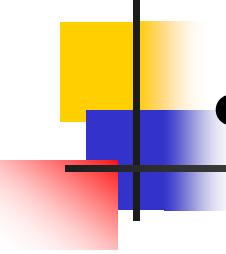
1.实验报告主题：基于深度学习的人脸识别系统

2.实验目的：

- 1) 了解Sphreface深度学习网络模型；
- 2) 了解Sphreface深度网络特征提取过程；
- 3) 把人脸检测模块加入工程中，并利用深度网络模型实现对面脸特征提取与识别的整个过程。

3.实验要求：

- 1) 熟悉深度学习工具的编程环境；
- 2) 熟练掌握深度学习网络结构及其设计；
- 3) 熟悉深度网络特征提取与人脸识别过程；



● 实验4

4.实验报告内容:

- 1) 利用OpenCV深度模型接口调用Sphreface模型，并实现对人脸图像的深度特征提取；
- 2) 把Fast-MTCNN人脸检测以及基于仿射变换的人脸矫正加入到人脸识别工程中，实现检测，矫正，特征提取与识别的整个流程；
- 3) 人脸数据集中有10个类别，在每个类中选取10张作业图片作为训练，并利用深度模型进行特征提取，最后计算出识别率。
- 4) 对实验预处理、参数调整、训练以及识别的各个环节作必要分析。

实验报告统一命名为：深大2022智能识别系统设计-学号-姓名-实验报告4.doc

如有提交程序，统一命名为：深大2022智能识别系统设计-学号-姓名-实验报告4-源代码.zip

截止日期2024.1.6下午5:00

● 实验4

 sphereface_deploy.prototxt
 sphereface_model.caffemodel

← <https://github.com/wy1iu/sphereface>

Network input

```

name: "SpherefaceNet-20"
input: "data"
input_dim: 1
input_dim: 3
input_dim: 112
input_dim: 96
##### CNN Architecture #####
layer {
  name: "conv1_1"
  type: "Convolution"
  bottom: "data"
  top: "conv1_1"

```

```

layer {
  name: "fc5"
  type: "InnerProduct"
  bottom: "res4_3"
  top: "fc5"
  param {
    lr_mult: 1
    decay_mult: 1
  }
  param {
    lr_mult: 2
    decay_mult: 0
  }
  inner_product_param {
    num_output: 512
    weight_filler {
      type: "xavier"
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}

```

Network Output

The extracted 512 dimensional features can be used for the recognition purpose

Deep feature extraction based on OpenCV

```
Net net;
string label_dir = TrainingSet_dir+"/Labels.txt";

//Specify Caffe Model
//modelTxt = proto_model_dir + "/face_deploy.prototxt";
//modelBin = proto_model_dir + "/face_model.caffemodel";

modelTxt = proto_model_dir + "/sphereface_deploy.prototxt";
modelBin = proto_model_dir + "/sphereface_model.caffemodel";

// Read Caffe Network
net = cv::dnn::readNetFromCaffe(modelTxt, modelBin);
if (net.empty())
{
    std::cerr << "Can't load network by using the following files: " << std::endl;
    exit(-1);
}

cv::String imageFile = "mytest.jpg";
cv::Mat image = cv::imread(imageFile, 1); // Color image
if (image.empty())
{
    std::cerr << "Can't read image from the file: " << imageFile << std::endl;
    exit(-1);
}
cerr << "image read successfully" << endl;

//Detect Face
CFaceDetectFMTCNN DetectObj;
vector<FaceInfo> faceInfo;
cv::Mat copyImg;
image.copyTo(copyImg);
DetectObj.DetectFace_FMTCNN(image, faceInfo);
//显示画图
DetectObj.DispFaceBoxLandMarks(copyImg, faceInfo);
cv::imshow("image", copyImg);

cv::Mat dstImage(IMG_NORHEIGHT, IMG_NORWIDTH, CV_8UC3);

for (int i = 0; i < faceInfo.size(); i++) //对每一个人脸进行处理
{
    FaceInfo faceInfoVar = faceInfo[i];
    DetectObj.RotateFace(image, faceInfoVar, dstImage);

    //Normalize Face
    cv::Mat normalizeImg;
    normalizeImg.create(dstImage.size(), CV_32FC3);
    DetectObj.NormalizeFace(dstImage, normalizeImg);
    //waitKey(0);

    //Construct the inputBlob
    //Mat inputBlob = blobFromImage(img_new, 1.0 / 255, Size(28, 28)); // 必须要归一化
    cv::Mat inputBlob = blobFromImage(normalizeImg);
    cv::Mat FeatureOutput; //Feature of the outputted inner_product layer
    net.setInput(inputBlob, "data"); //set the network input, "data" is the name of the input layer
    FeatureOutput = net.forward("fc5"); //compute output, "fc5" is the name of the output layer
    FeatureOutput.copyTo(faceInfo[i].FeatureVector);

    printf("FeatureOutput Size: rows=%d, cols=%d\n", FeatureOutput.rows, FeatureOutput.cols);
    for (int i = 0; i < FeatureOutput.rows; i++)
    {
        for (int j = 0; j < FeatureOutput.cols; j++)
        {
            printf("%f ", FeatureOutput.ptr<float>(i)[j]);
        }
        printf("\n");
    }
}
```

1. Read Caffe Network

2. Load Image

3. Detect Face

4. Face Calibration

- (1) Affine transform
- (2) Normalization
 - image value – mean value
 - Image values are set to 0~1

5. Input image data into network

6. Network forward and feature extraction

● 实验4

```

C:\Windows\system32\cmd.exe
net read successfully
image read successfully
Attempting to upgrade batch norm layers using deprecated params: model_FMTCNN/det1_.caffemodel
Successfully upgraded batch norm layers using deprecated params.
FeatureOutput Size: rows=1, cols=512
0.737988 0.467136 0.815362 0.302859 -0.616788 -1.558012 -0.258240 0.521465 0.413598 1.484955 0.328301 -0.938360 0.146698 -0.595733 -1.318992 -0.390480
39866 1.082312 -0.214120 0.391660 1.493435 1.695246 -0.797147 0.631210 -1.141055 -0.218179 -2.113643 0.198239 1.282373 1.372847 0.415100 0.618396 0.42
-0.683421 -0.587789 0.066951 -1.122008 -1.686354 0.438561 -1.929051 -0.982803 0.049263 -0.285489 1.030219 -0.842287 1.381375 -0.097213 -2.152589 1.39
7 -1.334017 1.430436 -1.037231 -0.237709 -0.594704 -0.932686 -0.942040 0.539077 1.720777 0.186546 0.927423 -1.369121 -1.362711 -2.448176 0.577782 0.19
.101385 1.630074 -0.927626 -0.833416 0.149907 0.387406 0.147509 -0.085059 -1.068083 0.573913 0.058556 -0.106980 -0.789087 -0.030413 -0.581654 -0.03551

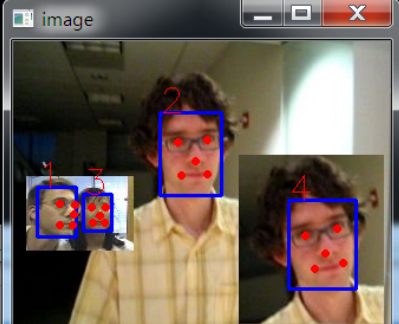
FeatureOutput Size: rows=1, cols=512
0.870203 -1.036407 2.120907 1.408023 -1.113631 -1.120334 2.258542 0.206351 1.384722 1.986325 0.260345 0.516912 0.616267 0.676256 0.715904 -0.481449 1.
0.626876 -0.755333 1.450257 0.023064 0.036600 -0.645490 -0.243868 0.037593 0.677166 0.324954 1.363754 1.695887 0.990943 0.648260 0.833509 1.234253 1.1
41 0.835476 -1.215726 0.484497 0.912230 2.517336 1.127420 0.942816 -1.470263 0.332503 -1.968051 -0.919297 -1.422325 1.112326 0.521988 0.268245 1.28008
0.955645 1.778423 1.342997 0.741092 1.321358 1.030583 1.120369 1.301635 0.082123 -2.220926 -0.042530 0.812977 2.124313 0.823139 1.735440 -0.320481 -0.
1690 -0.067079 1.685610 0.258100 -1.874013 -0.661040 -1.208561 1.836634 -0.341697 -1.591517 -1.471071 -0.882022 -0.657802 0.782041 0.885264 0.349180 -

FeatureOutput Size: rows=1, cols=512
-0.341046 1.173840 -0.284997 0.114773 -0.795273 -1.461071 -0.032344 -0.191543 -0.899895 1.738862 -1.985066 0.596693 -0.487101 -0.166614 -0.166493 -0.7
-0.495956 -0.816918 0.825081 -0.161961 -0.942687 0.911494 0.487532 0.154219 -1.353122 -0.297506 0.081390 -0.207490 0.153122 0.730842 1.148369 -0.43435
2 -0.625447 -0.979650 0.206616 -0.097144 -0.579534 -1.079345 0.451037 1.278493 0.121467 -0.166163 -0.051841 -0.052286 0.437007 0.374074 -0.068150 0.36
92364 0.007690 0.573613 0.606938 1.037387 0.184110 -1.479084 -0.693807 -0.418252 0.895489 -1.296831 -1.309113 0.833861 -1.583578 -1.881185 1.484902 0.
23956 -1.108914 -0.979264 -0.679285 -1.259886 -0.076051 -1.318762 0.334440 -2.732483 0.525584 0.518069 0.936838 -0.636252 0.722915 1.074739 -0.284417

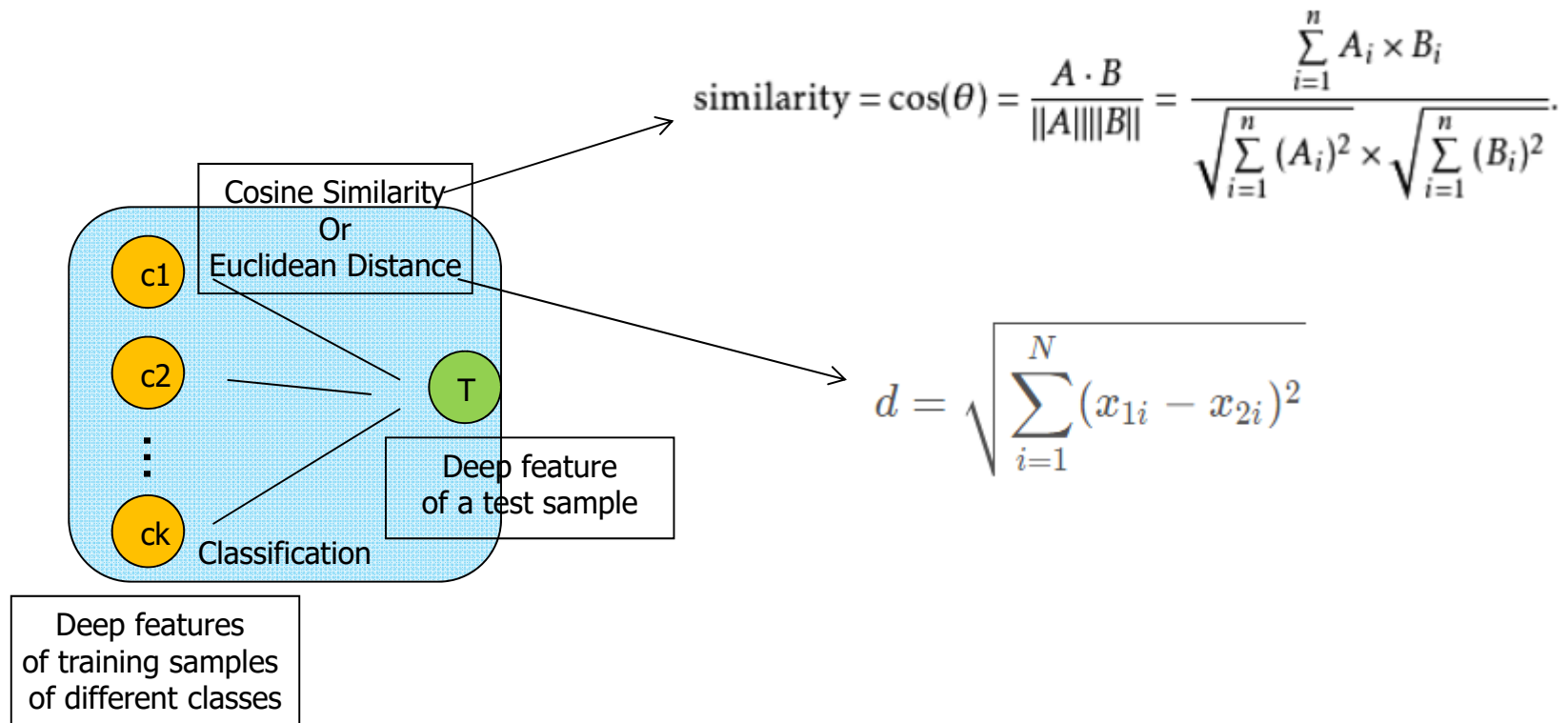
FeatureOutput Size: rows=1, cols=512
0.369094 -0.992173 2.057493 1.258057 -0.588400 -0.843643 1.887530 -0.339686 1.410960 1.730892 -0.020126 0.520423 0.198488 0.215064 0.832660 -0.179817
-0.102338 -0.575571 1.394984 0.109316 0.332147 -0.668637 -0.104353 0.393959 0.749644 0.536264 1.625471 2.051225 1.153362 0.893458 0.473560 1.211608 1
40969 0.921331 -0.765389 0.787280 0.940452 2.437143 1.079822 0.705991 -1.271256 -0.086626 -1.627314 -1.043271 -1.292778 1.375530 0.812736 0.333886 1.3
5912 -0.673463 1.906217 0.803460 0.533463 0.852739 0.660403 1.051296 0.972343 0.845556 -2.463035 0.007586 0.679836 1.906102 1.049898 1.885519 0.141462
8 1.637085 -0.149938 1.889664 0.386093 -1.548156 -0.728128 -1.340148 1.401345 -0.122680 -1.467855 -1.370691 -0.657638 -0.898547 1.283173 1.575053 0.35

u24=0.973754, u21=-0.074458, u23=0.055194

```



● 实验4



For cosine similarity, the maximum value indicates the best class.
For Euclidean distance, the minimum value indicates the best class.

● 实验4
